

lab1实验报告

实验基本信息

项目	内容
实验名称	Lab1：最小可执行内核
小组成员	2410673-韩羽宸、2410936-林子媛、2410933-马禹翔
完成日期	2026-10-05

本报告由小组三名成员共同完成，各模块按分工整理。

小组分工

成员	负责的练习/模块
2410673-韩羽宸	编译、链接、镜像生成和 QEMU 正常启动；分析 Makefile、链接脚本、ELF/bin 和 SBI 输出链
2410936-林子媛	入口代码、启动链分析与练习1
2410933-马禹翔	GDB 启动跟踪与练习2，以及提交前检查

韩羽宸负责构建与输出分析、运行验证和相关提示词；林子媛负责入口代码分析与练习1；马禹翔负责 GDB 启动跟踪与练习2。

一、实验目的

本实验主要围绕最小内核的构建与启动展开，构建与输出部分的目标如下：

- 使用 RISC-V 交叉工具链编译、链接源码，生成 ELF 文件和二进制内核镜像。
- 理解链接脚本怎样安排代码和数据的位置，以及内核入口与加载地址的关系。
- 使用 QEMU 和 OpenSBI 启动内核，观察实际运行结果。
- 分析从 SBI 字符输出到 `cprintf` 格式化输出的封装过程。

二、实验环境

实验在 Windows 的 WSL2 Ubuntu-22.04 中进行，源码目录为 `lab1/code_lab1`。

工具或环境	版本/用途
Ubuntu	22.04.5 LTS
GNU Make	4.3，用于组织构建
RISC-V GCC	<code>riscv64-unknown-elf-gcc</code> 10.2.0
系统 GNU ld	2.38，用于观察系统默认链接脚本
QEMU	7.0.0，模拟 RISC-V 计算机
OpenSBI	1.0；运行输出中的 Runtime SBI Version 为 0.3

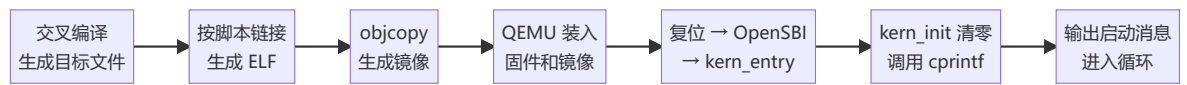
使用的 AI 工具如下：

成员	AI 编程工具	底层模型	备注
2410673-韩羽宸	OpenAI ChatGPT	GPT-6	辅助分析代码和定位启动问题

三、实验整体逻辑分析

2.1 本章节的逻辑主线

Lab1 将 C 和汇编源码编译为目标文件，按链接脚本生成 ELF，再转换为内核镜像。QEMU 将固件和镜像装入内存，CPU 经过复位代码、OpenSBI 和内核入口，最终执行 `kern_init`，输出启动消息并进入循环。



2.2 功能的逐步实现

按照手册顺序，先了解项目组成及 OpenSBI、ELF/bin，再分析链接布局 and SBI 输出封装，最后检查 Makefile 的构建过程并运行 `make qemu`。

现有代码已经包含上述功能，构建与输出部分主要进行构建、运行和分析，实际代码改动为 Makefile 中 `qemu` 目标的一项加载参数。

四、实验内容与实现

功能模块：构建与输出

负责人：2410673-韩羽宸

模块功能描述

需要实现/修改的函数：

本次沿用已有 C 函数，修改的是 Makefile 的 `qemu` 规则。分析中涉及的主要接口如下：

```
int cprintf(const char *fmt, ...);
int vprintf(const char *fmt, va_list ap);
void vprintfmt(void (*putch)(int, void *), void *putdat,
               const char *fmt, va_list ap);
void cons_putc(int c);
void sbi_console_putchar(unsigned char ch);
uint64_t sbi_call(uint64_t sbi_type, uint64_t arg0,
                  uint64_t arg1, uint64_t arg2);
```

功能说明：

1. 项目组成和执行流

`Makefile` 和 `tools/function.mk` 负责生成构建规则，`tools/kernel.ld` 负责安排链接后的内存布局。`entry.S` 设置栈指针，再跳转到 `init.c` 中的 `kern_init`。输出功能由 `stdio.c`、`printfmt.c`、`console.c` 和 `sbi.c` 共同完成。

`kern_init` 的主要操作如下：

```
memset(edata, 0, end - edata);
cprintf("%s\n\n", message);
while (1)
    ;
```

`edata` 和 `end` 由链接器定义，用于确定清零范围。随后输出启动消息，进入循环，这是当前最小内核的预期运行状态。

2. OpenSBI、ELF 和二进制镜像

QEMU 根据启动参数装入镜像，OpenSBI 在 M 模式完成底层初始化，再将控制权交给 S 模式内核；之后内核仍可通过 SBI 请求字符输出等服务。

`bin/kernel` 是包含入口、装入和调试信息的 ELF 文件；`bin/ucore.img` 是经 `objcopy` 转换得到的原始镜像，没有 ELF 头部，需要与加载配置配合。实际 `file` 检查分别显示 RISC-V 64 位 ELF 和 `data`。

`.bss` 通常只在 ELF 中描述所需空间，因此启动时仍需清零。

3. 内存布局、链接脚本和入口点

`ld -verbose` 展示的是宿主机默认脚本，架构为 x86-64，入口为 `_start`。内核实际使用 RISC-V 链接器和 `tools/kernel.ld`。

内核链接脚本的关键设置如下：

```

OUTPUT_ARCH(riscv)
ENTRY(kern_entry)

BASE_ADDRESS = 0x80200000;

SECTIONS
{
    . = BASE_ADDRESS;
    .text : {
        *(.text.kern_entry .text .stub .text.* .gnu.linkonce.t.*)
    }
    PROVIDE(etext = .);
}

```

这段代码节选自脚本开头。`OUTPUT\_ARCH` 指定架构，`ENTRY` 声明入口，位置计数器 `.` 从 `0x80200000` 开始，并优先收集入口代码。本次 ELF 的入口地址为 `0x80200000`，与成功启动时的下一阶段地址一致。

后续段分别保存只读数据、已初始化数据和零初始化数据。`ALIGN(0x1000)` 将数据起始位置对齐到 4096 字节边界；`edata` 位于初始化数据之后，`end` 位于 `.bss` 之后，两者确定清零范围。启动栈由 `entry.S` 预留并设置，具体入口操作由林子媛分析。

4. 从 SBI 到 stdio

本工程使用传统 SBI 接口，字符输出封装如下：

```

void sbi_console_putchar(unsigned char ch) {
    sbi_call(SBI_CONSOLE_PUTCHAR, ch, 0, 0);
}

```

输出服务编号为 1，字符作为第一个参数。`sbi\_call` 将服务编号放入 `a7`、参数放入 `a0` 等寄存器，通过 `ecall` 请求 M 模式 OpenSBI 服务，返回值从 `a0` 取出。

格式化输出的主要代码如下：

```

int vcprintf(const char *fmt, va_list ap) {
    int cnt = 0;
    vprintfmt((void *)cputch, &cnt, fmt, ap);
    return cnt;
}

```

`cprintf` 通过 `va\_start`、`va\_end` 管理可变参数，再调用 `vcprintf`。`vprintfmt` 解析格式，通过 `cputch` 回调逐字符输出并计数。完整调用链为：

```

kern_init → cprintf → vcprintf → vprintfmt
           → cputch → cons_putc → sbi_console_putchar
           → sbi_call → ecall → OpenSBI 控制台服务

```

这样，格式解析与底层输出分开，上层可以使用统一的 `cprintf` 接口。

5. Makefile 中的编译、链接和镜像生成

Makefile 使用 RISC-V 交叉工具链，`function.mk` 组织源码、编译和依赖规则，Make 根据依赖及更新时间决定是否重建。

链接和镜像转换的核心命令如下：

```
$(kernel): tools/kernel.ld

$(kernel): $(KOBJS)
    $(V)$(LD) $(LDFLAGS) -T tools/kernel.ld -o $@ $(KOBJS)

$(UCOREIMG): $(kernel)
    $(OBJCOPY) $(kernel) --strip-all -O binary $@
```

`$(KOBJS)` 汇集目标文件，`$@` 表示当前目标。链接器按脚本生成 `bin/kernel`，随后 `objcopy` 生成 `bin/ucore.img`。`-g` 保留 ELF 调试信息，`-nostdinc`、`-nostdlib` 使内核使用自己的头文件和基础实现；独立函数、数据节与 `--gc-sections` 配合，可删除未使用的节。

最终提示词

下面将启动修复任务整理为手册要求的四段式提示词，作为本模块的最终任务规格：

[PROMPT]

make 已经成功，但 `make qemu` 只打印了 `OpenSBI` 信息，没有出现内核消息。
请结合手册、Makefile 和链接脚本判断问题在哪个阶段。
原来的 `loader` 参数和 `-kernel` 参数有什么区别？确认原因后，直接修改 Makefile 的 `qemu` 规则，说明为什么这样改，以及怎么判断修复成功。

[RELY]

环境：Ubuntu 22.04、QEMU 7.0.0、OpenSBI 1.0。
原始参数：`-device loader,file=$(UCOREIMG),addr=0x80200000`。
异常输出：Domain0 Next Address 为 0，未出现内核启动消息。
镜像：`bin/ucore.img`；链接基址和当前入口地址为 `0x80200000`。
`kern_init` 打印“(THU.CST) os is loading ...”后进入循环。

[GUARANTEE]

只修改 Makefile 的 `qemu` 目标，将加载参数改为 `-kernel $(UCOREIMG)`。
保留编译、链接规则、`debug` 目标和内核源码，给出修改差异及复验依据。

[SPECIFICATION]

Pre-Condition: `bin/kernel` 和 `bin/ucore.img` 已成功生成。
Post-Condition: `OpenSBI` 下一阶段地址为 `0x80200000`，模式为 `S-mode`，随后输出内核消息。
Case 1: 只有固件信息时，检查下一阶段地址，不能直接判定内核已启动。
Case 2: 内核输出后停在循环中，符合当前 `kern_init` 的预期行为。
Requirements: 将结论限定在当前环境，区分装入地址与启动入口配置。

其他分析提示词见 [提示词汇总](#)。

实现迭代过程

本部分沿用已有内核，经历了首次构建运行和修改启动参数后的复验两个阶段。

第一次迭代

遇到的问题：

`make` 完成编译、链接和镜像生成，但执行原始 `make qemu` 后，只出现 OpenSBI 信息，没有内核启动消息。输出中的 `Domain0 Next Address` 为 `0x0000000000000000`，没有指向内核入口。

问题解决策略：

原来的 `-device loader` 指定了镜像装入地址，但在本机默认固件环境中没有正确完成下一阶段入口的配置。结合实际输出，将正常运行目标改为由 QEMU 的 `-kernel` 参数处理内核启动，具体改动为：

```
-      -device loader,file=$(UCOREIMG),addr=0x80200000
+      -kernel $(UCOREIMG)
```

实际只替换了一行参数，`debug` 目标由马禹翔继续验证。

第二次迭代

最终结果：

重新执行 `make qemu`，下一阶段地址变为 `0x0000000080200000`，模式为 `S-mode`，随后出现 `(THU.CST) os is loading ...`，说明已执行到内核输出位置。

关键改进点总结：

构建成功后，还需将固件的下一阶段地址与内核入口比较，单独验证启动交接。

练习：理解内核启动中的程序入口操作

负责人：2410936-林子媛

`la sp, bootstacktop` 与 `tail kern_init` 的详细解答由林子媛整理。

练习：使用 GDB 验证启动流程

负责人：2410933-马禹翔

从复位到内核入口的 GDB 跟踪记录及答案由马禹翔整理。

Challenge：本部分无额外 Challenge 任务

负责人：不适用

本部分无额外 Challenge。

五、测试与验证

测试截图：

按手册顺序展示默认链接脚本、构建和运行结果。

```
smile@smile:/mnt/e/Grade3_1/操作系统/os_lab/lab1/code_lab1$ ld --verbose
GNU ld (GNU Binutils for Ubuntu) 2.38
Supported emulations:
  elf_x86_64
  elf32_x86_64
  elf_i386
  elf_iamcu
  elf_llom
  elf_klom
  i386pep
  i386pe
using internal linker script:
=====
/* Script for -z combrelloc -z separate-code */
/* Copyright (C) 2014-2022 Free Software Foundation, Inc.
   Copying and distribution of this script, with or without modification,
   are permitted in any medium without royalty provided the copyright
   notice and this notice are preserved.  */
OUTPUT_FORMAT("elf64-x86-64", "elf64-x86-64",
              "elf64-x86-64")
OUTPUT_ARCH(i386:x86-64)
ENTRY(_start)
```

图1: `ld --verbose` 输出及默认链接脚本开头。

`make` 完成编译、链接和镜像生成，文件类型检查也确认两种产物已生成。

```
smile@smile:/mnt/e/Grade3_1/操作系统/os_lab/lab1/code_lab1$ make
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/driver/console.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img
```

图2: `make` 完成编译、链接和镜像生成。

`make qemu` 输出的固件基址为 `0x80000000`，下一阶段地址为 `0x80200000`，随后出现内核消息并进入循环。

AI 协作开发的经验

小组成员执行命令并提供实际输出，AI 辅助分析代码和定位问题。将固件的下一阶段地址与链接脚本对应后，修复范围缩小到一项启动参数。修改后仍需重新运行验证，才能确认分析是否成立。
